System Design of WhatsApp / Chat Application

Functional Requirements (What the user should be able to do)

1. User Registration, Authentication, Profile Management
2. User Profile Picture management (Optional)
3. 1 on 1 Chats, Group Chats (up to 100 members)
4. Messages should be delivered in real time if the receiver is online.
5. Messages should be stored and delivered later if the receiver is offline.
6. Support media in messages.
7. Online / Last Seen Status (optional)
8. Multi-Device Support (Optional)

Non-Functional Requirements

1. Handle 100M DAU, Few Billion users -> High throughput
2. Messages should reach the receiver in near real-time (less than 500 ms)
3. Availability >> Consistency
4. Fault tolerance (Resilient)
5. Messages not stored unnecessarily (WhatsApp)

Core Entities

1. User
2. Chat
3. Message

APIs

1. POST /api/v1/users

```
{
  username, email etc
}
```

2. GET /api/v1/users/{user_id}

WebSocket Commands

Endpoint: /api/v1/chat/ws

3. Create Chat

Client → Server

```
{
  "command": "CREATE_CHAT",
  "payload": {
    "members": [],
    "name": ""
  }
}
```

Server → Client

```
{
  "chatId": "chat_123"
}
```

4. Send Message

Client → Server

```
{
  "command": "SEND_MESSAGE",
  "payload": {
    "chatId": "chat_123",
    "text": "",
    "attachmentIds": []
  }
}
```

Server → Client

```
{
  "status": "SUCCESS",
  "messageId": "msg_456"
}
```

5. Add / Remove Chat Members

```
{
  "command": "UPDATE_CHAT_MEMBERS",
  "payload": {
    "chatId": "chat_123",
    "memberId": "user_9",
    "operation": "ADD"
  }
}
```

6. New Message Event

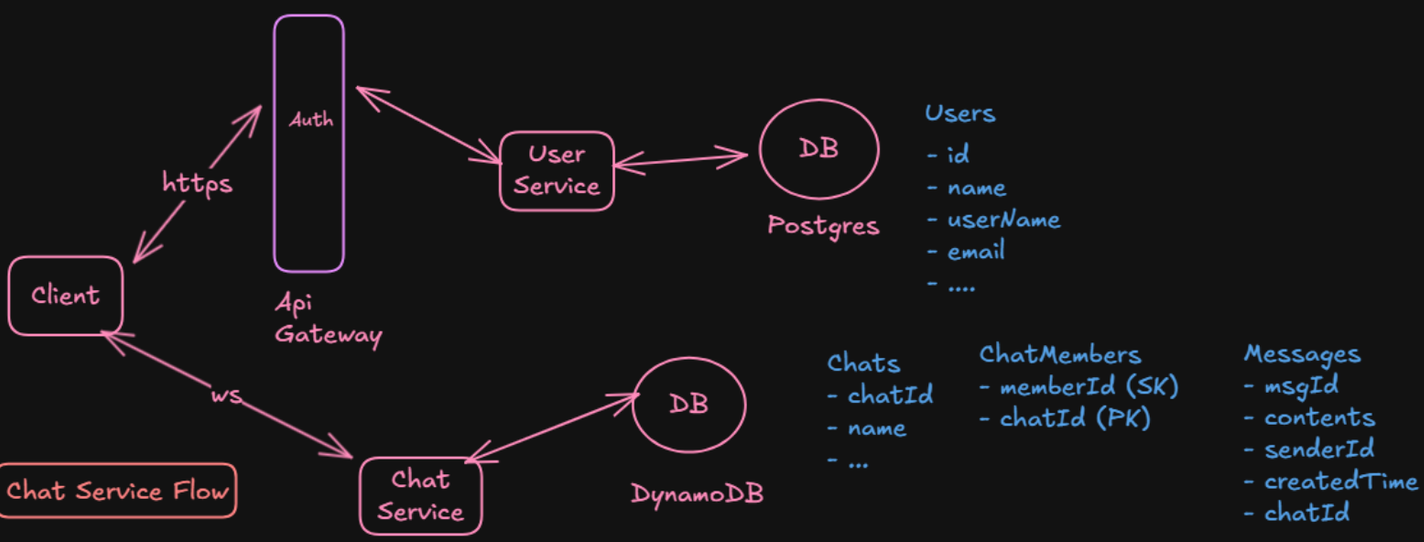
Server → Client

```
{
  "event": "NEW_MESSAGE",
  "eventId": "evt_2",
  "payload": {
    "chatId": "chat_123",
    "senderId": "user_1",
    "text": "",
    "attachmentIds": []
  }
}
```

Client → Server (ACK)

```
{
  "command": "ACK",
  "eventId": "evt_2"
}
```

High Level Design



Connection Model

-> Client opens a WebSocket connection to the Chat Service
-> This connection is maintained for:

Chat creation
Messaging
Membership updates

Chat Creation Flow

-> Client sends a createChat command over WebSocket
-> Chat Service receives the request
-> Chat Service performs the following atomically:
  -> Creates a new Chat record
  -> Creates a ChatMember entry for each user

-> Chat Service returns the generated chatId to the client

Why This Must Be Atomic
  -> Chat without members is useless
  -> Members without a chat is invalid
  -> Both must succeed or both must fail

To get all members of a chat, we do it by querying by chatId(PK) -
this will be fast since :
  -> DynamoDB physically groups items by Partition Key
  -> All rows with the same chatId live together on disk

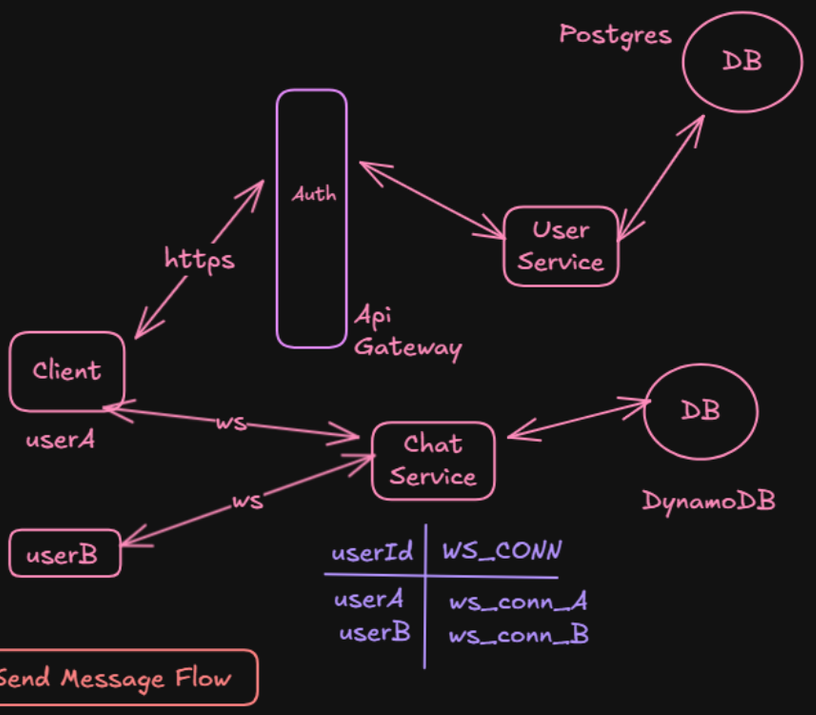
To get get all chats for a user, we need GSI

The GSI reorganizes the same data with a new key layout:

GSI
PK = memberId
SK = chatId

Functional Requirements (What the user should be able to do)

1. User Registration, Authentication, Profile Management
2. User Profile Picture management (Optional)
3. 1 on 1 Chats, Group Chats (up to 100 members)
4. Messages should be delivered in real time if the receiver is online.
5. Messages should be stored and delivered later if the receiver is offline.
6. Support media in messages.
7. Online / Last Seen Status (optional)
8. Multi-Device Support (Optional)



Send Message Flow

-> Client sends message over an existing WebSocket connection with chatId
-> Chat Service queries ChatMembers table using chatId (partition key)
-> Chat Service gets list of all members in the chat
-> Chat Service maintains an in-memory map:
userId → WebSocket connection

For each member:
-> Look up active WebSocket connection
-> Send message if user is online
-> Clients send ACK to confirm delivery

Offline Message Delivery (When user is currently offline)

We introduce a table called PendingDeliveries:
-> One entry per (recipient, message) pair
-> Acts as a temporary holding area until delivery is confirmed
-> Sender sends a sendMessage command to the Chat Server
-> Chat Server fetches all members of the chat
-> Chat Server:
In one DynamoDB transaction:
-> Write to Messages
-> Write to PendingDeliveries (for all recipients)

-> so that upto 100 records (group limit) easily supported by dynamoDB in a single transaction
-> Chat Server responds to sender with success/failure and message ID
-> Chat Server attempts real-time delivery over WebSockets

If a recipient is online:
Message is pushed immediately
Client sends an ACK
Corresponding PendingDeliveries entry is deleted

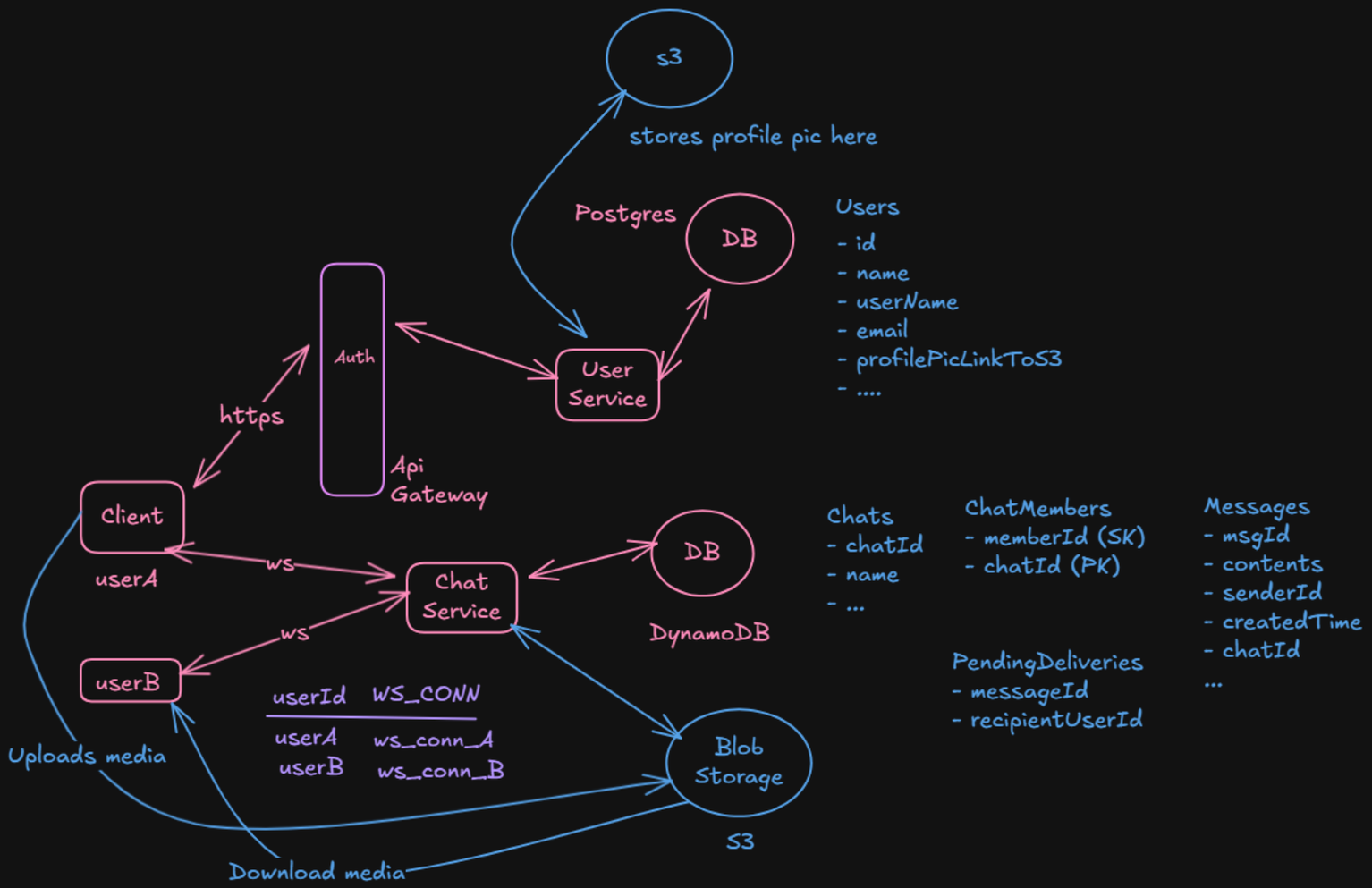
If a recipient is offline:
Message remains in PendingDeliveries
No delivery attempt is required

Reconnect Flow (Offline Users)

When a user comes back online:

-> Chat Server queries PendingDeliveries for that user
-> Retrieves all pending message IDs
-> Fetches full message content from Messages
-> Sends messages to the client
-> Client ACKs each message
-> Chat Server deletes delivered entries from PendingDeliveries

We can also delete undelivered msgs after 30 or 60 days may be if recipient doesnt come online



Support media in messages

Why Media Is Not Stored in DB

-> Media files are large and unstructured
-> Databases are optimized for small, structured records
-> Storing blobs increases cost and slows queries
--> DB is used only for metadata, not media

Why Media Should Not Pass Through Chat Server

-> Chat Server handling uploads/downloads becomes a bottleneck
-> Consumes bandwidth, memory, and CPU
-> Slows down real-time messaging
--> Chat Server should stay lightweight

Why S3 + Pre-Signed URLs Works

Sender Flow

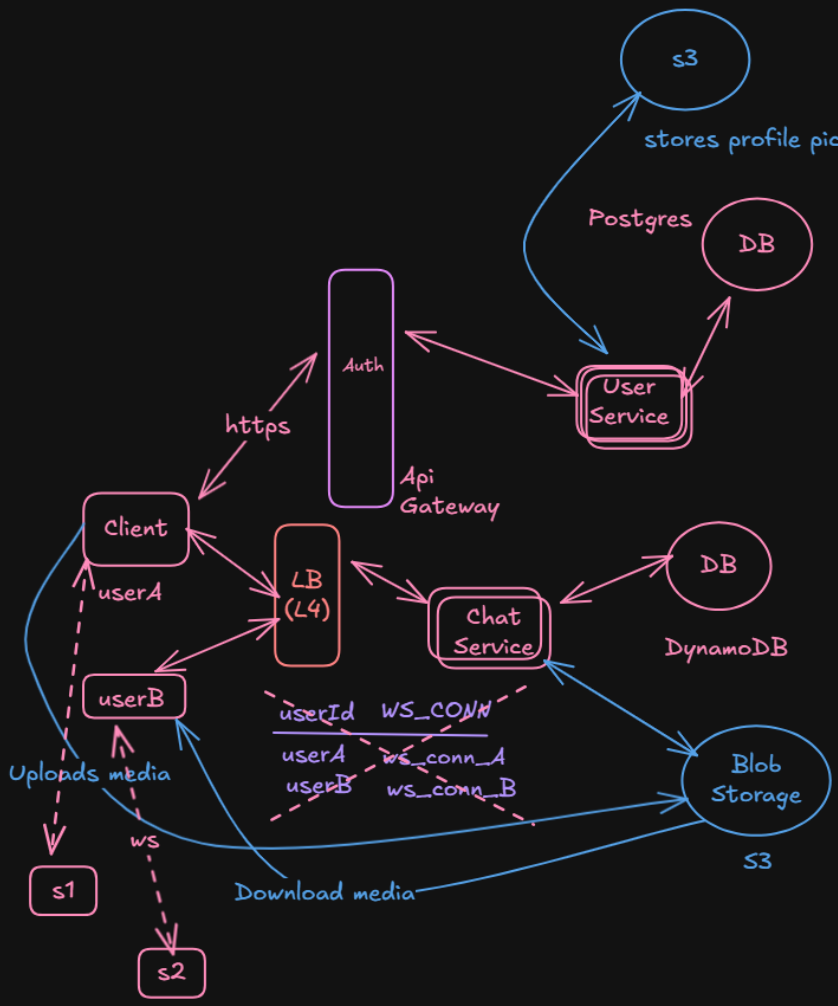
-> Sender requests an upload target from Chat Server
-> Chat Server returns a pre-signed S3 upload URL
-> Sender uploads media directly to S3
-> Sender sends a normal chat message containing:
    chatId
    media reference

Server Flow

-> Chat Server stores the message (text + media reference)
-> Chat Server creates pending delivery records
-> Chat Server attempts real-time delivery to online users

Receiver Flow

-> Receiver gets the message over WebSocket
-> Message contains the media reference
-> Receiver uses a pre-signed download URL to fetch media directly from S3
-> Receiver ACKs message delivery



Kafka — User based topics

Deep Dives

Handle Large Scale (Billions Users)

We can add more chat servers, but then
how server will know which server the recipient is connected to ?

Solution 1: Use Kafka -> Topic per user

not wrong, but bad

-> Kafka topics are heavy
-> Metadata overhead per topic is large
-> billions of users = Huge topic count (kafka is not good for this)

Kafka is great for:
- Streams
- Logs
- Analytics

Kafka is not for:
- Fine-grained fan-out per user

Interview takeaway:

"Using Kafka topics per user is an anti-pattern."

Load Balancer Choice

We place the Chat Service behind an L4 load balancer.

Why L4 and not L7?

-> WebSocket is a long-lived TCP connection
-> We don't need:

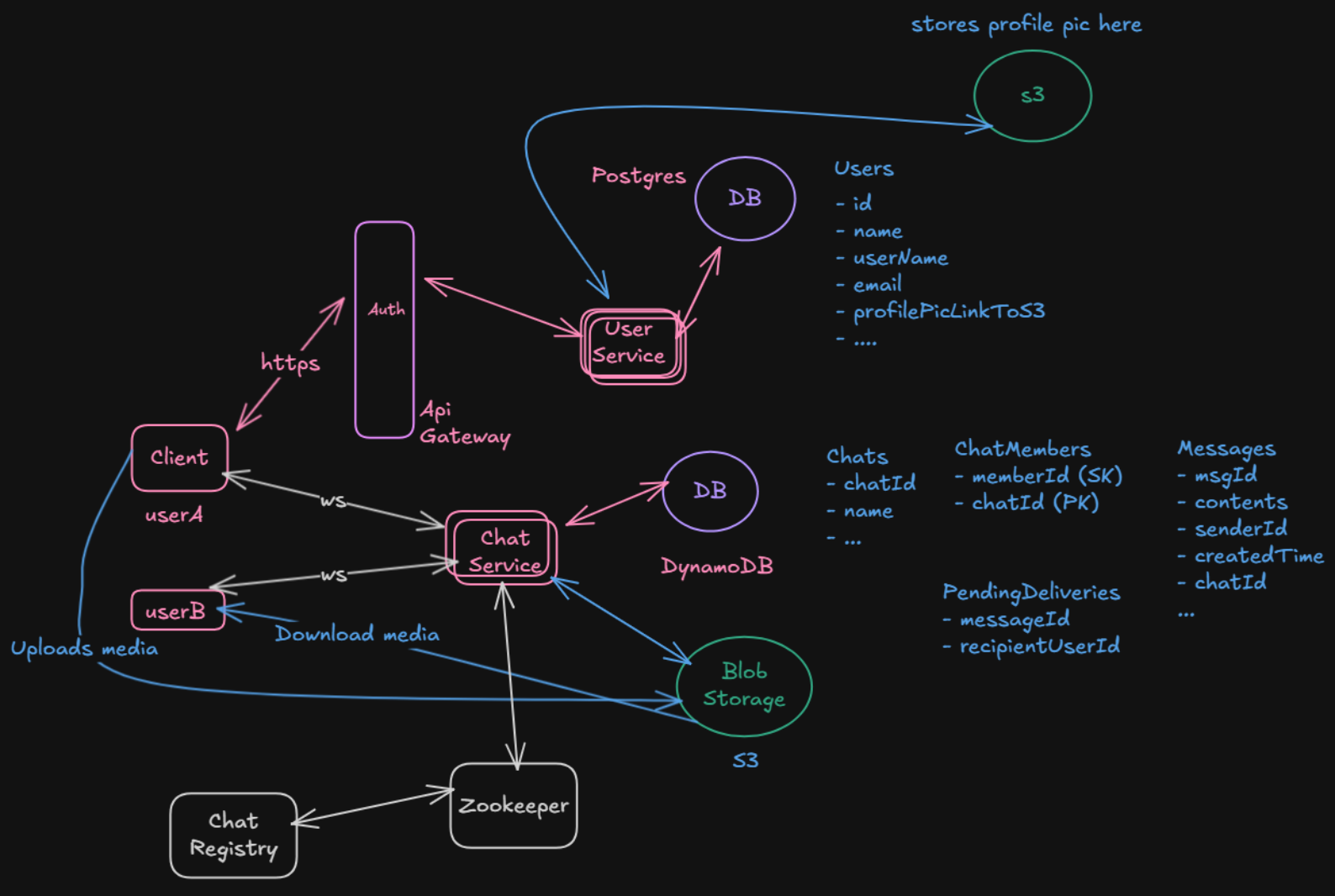
- Path-based routing
- Header-based routing
- HTTP-level inspection

->All chat traffic goes to the same service

L4 is:

- Simpler
- Faster
- Lower latency

Perfect for persistent connections



Solution 2: Consistent hashing of chat servers With Zookeeper or Etcd

Improved Solution

PHASE 0: System bootstrapping

0.1 Chat Servers start

Each Chat Server (S1, S2, S3):
Generates its hash tokens (virtual nodes)
Registers itself in ZooKeeper

0.2 ZooKeeper now knows

Which chat servers exist
Which hash positions they own
Which servers are alive (ephemeral nodes)

0.3 Chat Registry starts

Chat Registry:

Watches /chat-servers/* in ZooKeeper
Reads all servers + tokens
Builds a consistent hash ring locally in memory

0.4 Chat Servers also build the same hash ring

Each Chat Server:

Watches ZooKeeper
Reads the same server/token data
Builds the same hash ring locally

At this point:

-> Chat Registry and all Chat Servers have identical hash rings

PHASE 1: User A opens the app

1.1 User A has no server yet
User A → Chat Registry (HTTPS)

Message:
"I am user A. Which server should I connect to?"

1.2 Chat Registry decides using hash ring

Chat Registry:
Computes hash(userId_A)
Looks up the hash ring
Finds owner server (say S2)

1.3 User A connects to S2
User A → S2 (WebSocket / TCP)

Authentication happens.
Persistent connection established.

-> User A is now online on S2.

PHASE 2: User B opens the app

Same flow:

User B → Chat Registry
hash(userId_B) → S3
User B → Chat Server S3

Now:

User A → S2
User B → S3

PHASE 3: User A sends a message to User B

3.1 Message arrives at S2
User A → S2

Payload:
(from=A, to=B, content="Hi")

3.2 S2 persists message

PHASE 4: S2 finds where User B lives

4.1 S2 computes owner of B

S2 already has the hash ring locally.
hash(userId_B) → falls in S3's range
So:

owner(B) = S3

PHASE 5: Server-to-server delivery

S2 → S3 (internal RPC/TCP)
S3 already has an open connection to User B.

PHASE 6: Message delivered to User B

S3 → User B (WebSocket)

Cleanup

PHASE 7: Failure handling

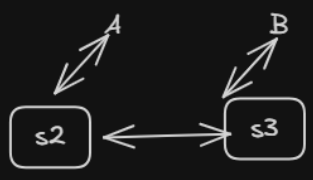
If S3 crashes

ZooKeeper session for S3 expires
/chat-servers/S3 disappears
Watchers fire on:
Chat Registry
Chat Servers

Hash ring is rebuilt
User B reconnects
New owner server is assigned

ZooKeeper = shared truth about servers
Chat Registry = initial placement
Chat Servers = hashing + routing
Hash Ring = deterministic math

-> Chat servers register their hash positions in ZooKeeper.
-> Both chat registry and chat servers watch ZooKeeper and
build identical consistent hash rings locally.
-> When a client connects, the registry hashes the userId and
returns the owner server.
-> When a message is sent, the sender's server hashes the recipient's userId
to find the owner server and forwards the message directly.



Major Problems With this Solution

Problem 1: Server-to-server connection explosion

"Each Chat Server will need to maintain connections with each other Chat Server"

Problem 2: Scaling requires careful orchestration

"Increasing the number of Chat Servers requires careful orchestration"

Why

- Adding/removing servers changes the hash ring
- User ownership changes
- Users must move to new servers

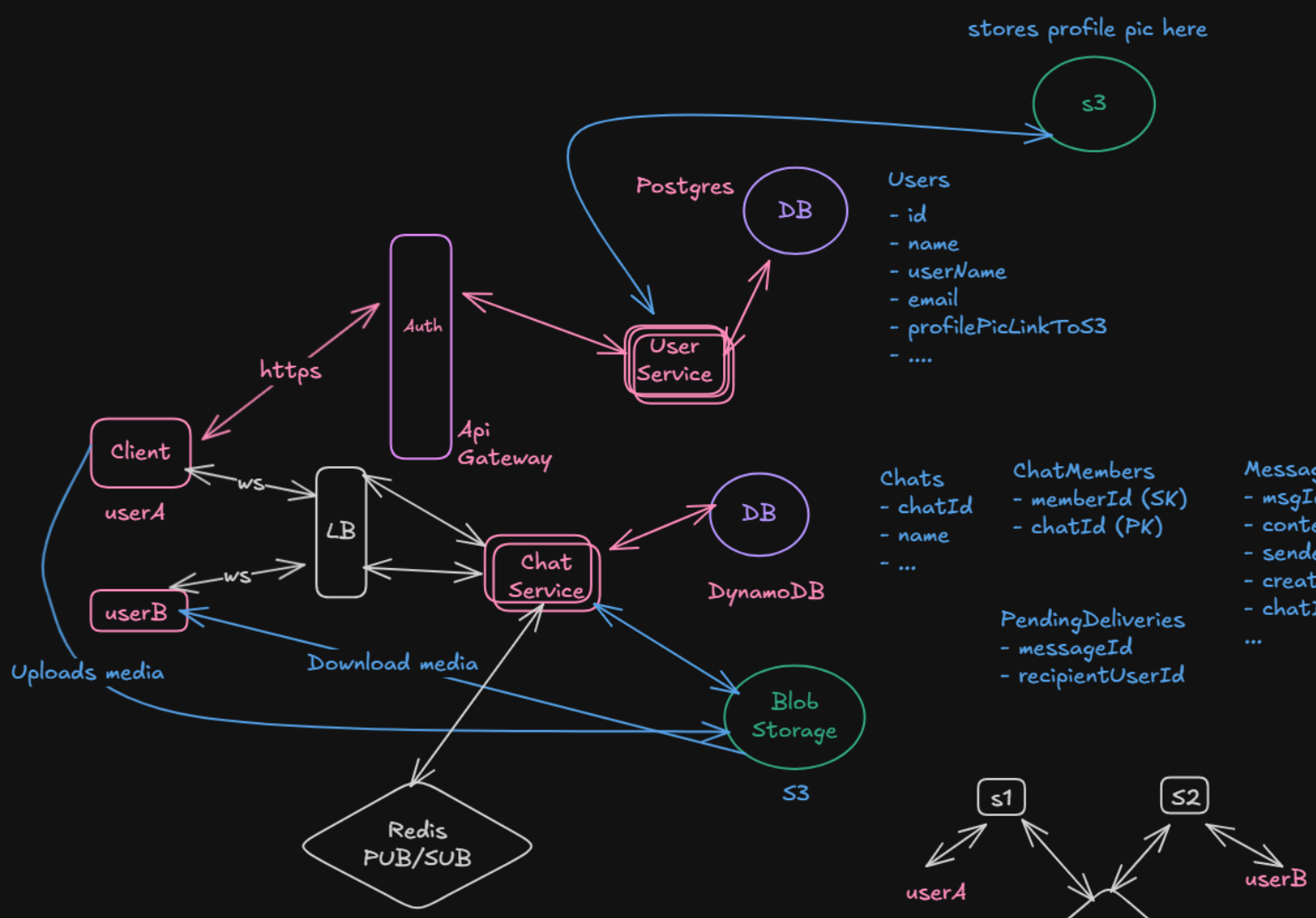
This causes:

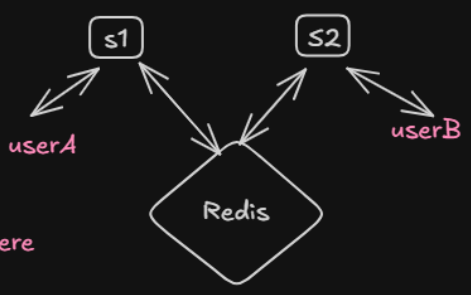
- Reconnect storms
- Load spikes

Problem 3: Message loss risk during migration

"During scaling we need to ensure that events are sent to both servers"

Problem 4: Hot users & uneven load





Solution 3 (Best): Use Redis PUB/SUB -> Topics are very lightweight here

User A sends message to Chat Server

Chat Server writes message to DB and creates PendingDeliveries entry

Chat Server sends ACK to User A

Chat Server publishes message to Redis Pub/Sub (receiver's channel -> userId topic)

Redis delivers message to subscribed Chat Server

Chat Server forwards message to User B via WebSocket

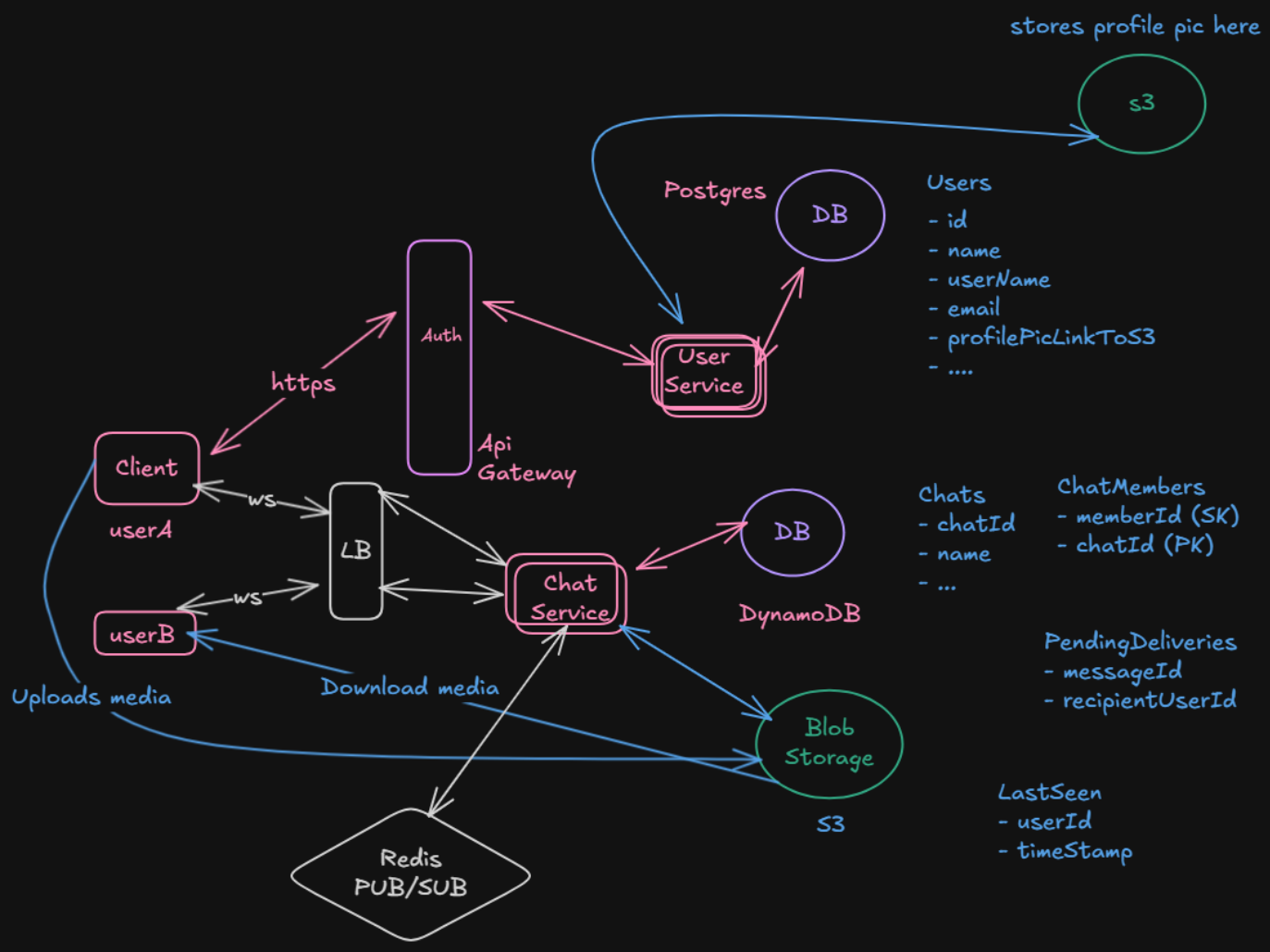
If User B is offline, message stays in DB and is delivered on reconnect

-> No complex ownership / rehashing logic

-> Scaling Chat Servers is MUCH easier

With Pub/Sub:
- New Chat Server starts
- Subscribes to user topics
- Starts receiving messages immediately

-> No user migration logic.
-> No special handling.



Last Seen / Online Flow

Core principles

-> ONLINE is never stored
-> Only timestamps are stored
-> Presence is calculated on request
-> DB = fallback truth
-> Chat Server = real-time truth

STEP 0: Connection + Heartbeat lifecycle

When user CONNECTS

WebSocket connection established
Chat Server marks user as online (in memory)
Write to LastSeen table (safety net):

LastSeen[userId] = now
(only if now > existing value)

Why connect-write?

Previous server may have crashed
Disconnect event may be missing
This self-heals stale timestamps

Heartbeat flow

Client sends heartbeat every N seconds (e.g. 20s)
Chat Server tracks last heartbeat timestamp in memory
No DB write on heartbeat

Purpose of heartbeat:

Detect silent disconnects
Handle app kill / network loss
Decide when to close connection

When heartbeats STOP

If:
No heartbeat for 2 × interval (e.g. 40–60s)

Then:
Server assumes user is disconnected
Server closes WebSocket
Server writes:

LastSeen[userId] = now
(conditional update)

--> This is the main and most common disconnect path

Explicit disconnect

App closed / socket closed
Same handling as heartbeat timeout
Write LastSeen timestamp

STEP 1: User A asks for last seen of User B

Client sends:

```
getLastSeen {
  targetUserId: B,
  requestingUserId: A
}
```

Triggered when:

Opening chat
Viewing profile

STEP 2: Chat Server receives request

Chat Server does NOT wait to know if B is online.

It does two things in parallel.

STEP 3: DB lookup (fallback path)

Read:
LastSeen[B]

Immediately send to User A:

```
updateLastSeen {
  reporter: DATABASE,
  lastSeen: <timestamp>
}
```

Why send immediately?

DB is guaranteed
Pub/Sub may not respond
Client should never block

STEP 4: Live online check via Redis

In parallel, server publishes:
getLastSeen → channel:user:B

Meaning:
"If B is online, reply."

STEP 5: If User B is ONLINE

B's Chat Server is subscribed to channel:user:B
Receives getLastSeen

Confirms user is still connected
Publishes:

```
updateLastSeen {
  reporter: SERVER,
  lastSeen: ONLINE
}
```

To:
channel:user:A

STEP 6: Client merges responses

Client logic:

Received	UI
ONLINE	Show "Online"
Only timestamp	Show "Last seen at X"

ONLINE always overrides DB response.

Client may:

Wait ~200ms before rendering, or
Update UI smoothly

STEP 7: If User B is OFFLINE

Only DB response exists
Client shows:
Last seen at <timestamp>

STEP 8: Crash scenarios (why heartbeats + connect-write matter)

Server crashes

Heartbeat stops
Client reconnects
New server writes connect timestamp
Data self-heals

Exact offline time lost?

Yes, sometimes
Accepted by design

We store:
"Last time we are sure the user was active"

Not:
"Exact second user went offline"

What if a WebSocket "looks" connected but actually isn't?

Mobile networks are unreliable.
A WebSocket can be open, even though the phone has:

Lost signal
Switched networks
Gone into sleep mode

The server won't immediately know.
TCP may take minutes to realize the connection is dead — far too slow for chat.

-> So we cant rely on TCP alone to find out when the connection breaks

Solution :Heartbeats

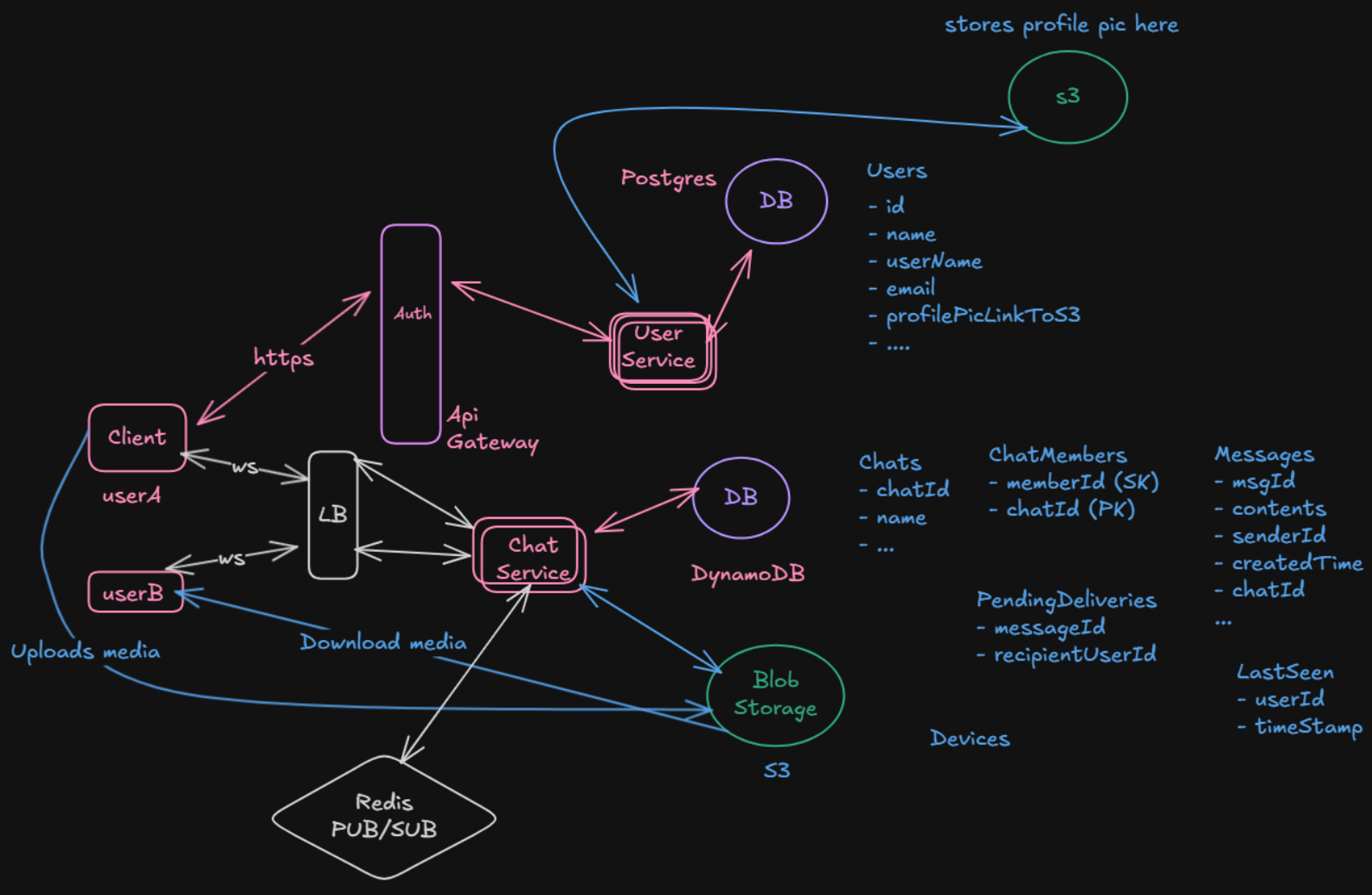
How it works

-> Server sends a small ping every 10–20 seconds
-> Client must reply with pong
-> If no pong arrives within a short timeout:

-> Server closes the connection
-> Client reconnects
-> Missed messages are synced from PendingDeliveries

Why this is ideal

-> Detects dead connections even during idle periods
-> Failure is caught in seconds, not minutes
-> Makes presence and delivery reliable



Handling multiple devices per user

A user can be logged in from multiple devices (phone, laptop, tablet)

We must deliver messages to each device independently

Changes needed

-> Add a Device table
-> Change PendingDeliveries from per-user to per-device
-> When sending a message:

-> Find all users in the chat
-> For each user, find all their devices
-> Deliver message to all devices

What does NOT change

-> Redis Pub/Sub remains per user
-> Chat servers still subscribe using userId
-> No change in Pub/Sub design

Cleanup & limits

-> Remove devices when they log out
-> Expire inactive devices
-> Limit number of devices per user (e.g. max 3 to 5)

How Messages ordering is handled ?

-> All chat servers keep their system time in sync with a trusted time source, so they all agree on what time it is, within a small error margin.
-> When a message reaches a chat server, the server records the current time
-> This timestamp becomes the message's display time
-> Clients sort messages based on this server-assigned time

As a result:

-> All devices show messages in the same order
-> Ordering is consistent, even if delivery paths differ

What users experience

A message may appear slightly above or below where it was expected

This happens rarely and is generally acceptable, since:

Messages arrive quickly
Conversation flow still feels natural

User-based vs Chat-based Pub/Sub channels

You can design Redis Pub/Sub channels in two ways:
-> One channel per user
-> One channel per chat (group)

Which one is better depends on how people chat.

Case 1: Mostly 1-to-1 chats (WhatsApp-like)

Example

Each user talks to 200 people
All chats are private (2 people)

User-based channels

1 user → 1 channel
Server subscribes to 1 channel
Message publish = 1 publish

Chat-based channels

200 chats → 200 channels per user
Server must subscribe to 200 channels
Lots of unused, duplicate channels

--> User-based is clearly better

Case 2: Very large group chat

Example

One group with 800 users
Everyone is chatting there

User-based channels

One message → 800 publishes
Expensive and noisy

Chat-based channels

One group → 1 channel
One publish → all users get it

--> Chat-based is clearly better

So we used User-based for our WhatsApp Design

Handling rare large groups (advanced idea)

Smart hybrid approach

Small chats → user channels
Large chats → chat channels

On connect:

-> Subscribe to your user channel
-> Also subscribe to any large group channels (we can set some limit to call large)

On send:

-> If group is large → publish once to group channel
-> Else → publish to user channels

Why this works

Cheap for normal chats
Efficient for big groups
No unnecessary Redis load

User channels are best for private chats,
chat channels are best for large groups — real systems combine both.

What if real-time delivery via Redis misses a message?

Redis Pub/Sub has at-most-once message delivery semantics.
This means a message is delivered once if at all,
and there are no retries or guarantees of delivery.

-> well we are saving msgs to DB , so they will be delivered some time later.
but if we want to deliver msgs to clients faster , then we need some solution

Solution 1: Periodic catch-up checks

Idea

Every connected client occasionally asks:
"Did I miss anything?"

The server checks the PendingDeliveries
Any undelivered messages are sent immediately

Trade-off

Simple and reliable

But creates steady background traffic

At very large scale, frequent polling increases server load.
Reducing polling frequency lowers load but increases delay.

--> This method works, but we need improvement

Solution 2: Detect gaps using message counters

Idea

-> Each chat assigns messages increasing numbers (1, 2, 3, ...)
-> Clients remember the last number they received
-> If they suddenly see message no.8 after no.6:
They know no.7 was missed
They request a sync

Benefits

-> No constant polling
-> Sync only happens when something is missing

Limitation

-> If there is no new msg after the missed message, missed msg is never noticed

Solution 3 (Preferred): heartbeats + sequence tracking

Idea

-> Instead of tracking counters per chat, track them per user.

-> Maintain a single increasing counter for each user
-> Every message delivered to that user increments the counter
-> Heartbeat messages from server include the current counter value
-> Client compares it with what it has locally
-> If behind → immediately requests a sync

Why this works well

-> Heartbeats already exist, so no extra traffic
-> Missing msgs are detected quickly
-> Covers missed messages from any chat, not just one

Cost and complexity

-> Requires an atomic counter per user
-> Usually implemented with Redis INCR
-> Adds a small coordination dependency

But in return:

-> Detection is fast
-> Load stays low
-> Polling becomes rare, not constant